

Fase 4

María Alexandra Jiménez Suárez

Juan José Álvarez Restrepo

Carolina Ramírez Lotero

Natalia Giraldo Morales

Universidad Pontificia Bolivariana

Facultad de Ingeniería

Cesar Augusto López Gallego

10/08/2026

Justificación de la elección de SC-1

De las tres solicitudes de cambio analizadas en la Fase 2, se eligió SC-1 (incorporar cosméticos y productos comestibles) porque es la que mejor se conecta con la jerarquía de productos que ya existía en el sistema. Tanto los medicamentos como los cosméticos y comestibles comparten los mismos atributos base (nombre, precio, stock, stock mínimo, fecha de vencimiento), por lo que extender Producto con nuevas subclases era una solución natural, sin forzar el modelo.

SC-2 (servicios de farmacia) se descartó para esta fase porque un servicio no tiene stock físico ni fecha de vencimiento real, así que habría exigido separar primero el concepto de "producto" del de "servicio" antes de poder demostrar cualquier principio, lo cual es un cambio más profundo que el que da tiempo de implementar y probar correctamente. SC-3 (convenios y descuentos) tampoco se centra en la creación de productos, sino en reglas comerciales sobre el cliente, por lo que no era la solicitud más directa para evidenciar OCP a través de la jerarquía ya diseñada.

Por esa razón, SC-1 fue la opción que permitió mostrar de forma más clara y medible que el sistema quedó abierto a extensión sin tener que modificar la lógica de negocio ya existente.

1. Observaciones en el nuevo código implementando buenas prácticas de S.O.L.I.D.

En el archivo original ServicioProducto.cs, CargarDesdeArchivo() llamaba directamente File.Exists(ruta) y File.ReadAllLines(ruta), y adentro del mismo método construía new MedicamentoCapsula(...).

```
public string CargarDesdeArchivo(
    string ruta)
{
    try
    {
        if (!File.Exists(ruta))
        {
            return "Archivo no encontrado";
        }

        string[] lineas =
            File.ReadAllLines(ruta);

        foreach (string linea in lineas)
        {
            string[] datos =
                linea.Split(';');

            Laboratorio laboratorio =
                new Laboratorio(
                    datos[5],
                    "Medellin",
                    "44444444");

            MedicamentoCapsula medicamento =
                new MedicamentoCapsula(
                    datos[0],
                    decimal.Parse(datos[1]),
                    int.Parse(datos[2]),
```

Y aquí se violaba DIP, ya que un módulo de alto nivel (la lógica de negocio de productos) dependía directamente de un módulo de bajo nivel.

Por lo que se solucionó en la creación de la interfaz IRepositoryProducto (con Existe() y LeerProductos()) y la clase RepositorioMedicamentoArchivo que la implementa. ServicioProducto

ahora recibe IRepositoryProducto **por constructor** y solo llama repositorio.Existe(...) / repositorio.LeerProductos(...).

```
public interface IRepositoryProducto
{
    bool Existe(string ruta);

    IEnumerable<Producto> LeerProductos(string ruta);
}
```

```
public class RepositorioMedicamentoArchivo : IRepositoryProducto
{
    2 references
    public bool Existe(string ruta)
    {
        return File.Exists(ruta);
    }

    2 references
    public IEnumerable<Producto> LeerProductos(
        string ruta)
    {
        string[] lineas = File.ReadAllLines(ruta);
    }
}
```

En el diseño original, ServicioProducto dependía directamente de File y construía el tipo concreto MedicamentoCapsula, violando DIP. En el rediseño, se introdujo la abstracción IRepositoryProducto; ServicioProducto solo depende de esa interfaz y recibe la implementación por constructor. Esto permite cambiar el origen de los datos sin tocar la lógica de negocio.

En el archivo ServicioCliente, también introdujo la abstracción de IRepositoryCliente

```
public class ServicioCliente
{
    4 references
    private List<Cliente> clientes;
    3 references
    private readonly IRepositoryCliente repositorio;
    2 references
    private readonly INotificadorPuntos notificadorPuntos;

    1 reference
    public ServicioCliente(
        IRepositoryCliente repositorio,
        INotificadorPuntos notificadorPuntos)
    {
        clientes = new List<Cliente>();
    }
}
```

```
public interface IRepositoryCliente
{
    bool Existe(string ruta);

    IEnumerable<Cliente> LeerClientes(string ruta);
}
```

Otro comportamiento encontrado en el código original, fue la violación DIP, el servicio quedaba "fijado" (acoplado) a esa clase concreta de evento. Y no se podía sustituirla por otra manera de notificación sin editar el servicio.

```

public ServicioProducto()
{
    productos = new List<Producto>();

    EventoStock = new EventoStockMinimo();
    EventoVencimiento = new EventoVencimiento();
}

```

En ServicioProducto, el constructor original hacía `EventoStock = new EventoStockMinimo();` y `EventoVencimiento = new EventoVencimiento();`.

```

public interface INotificadorStockMinimo
{
    void NotificarStockMinimo(Producto producto);
}

```

```

public interface INotificadorVencimiento
{
    0 references
    void NotificarVencimiento(Producto producto);
}

```

Entonces, como solución aplicada: se crearon 4 interfaces mínimas : `INotificadorStockMinimo`, `INotificadorVencimiento`, `INotificadorPuntos`, `INotificadorMovimiento`,

```

public class EventoStockMinimo : INotificadorStockMinimo
{
    1 reference
    public delegate void DelegadoStock(
        string mensaje);

    2 references
    public event DelegadoStock? StockMinimo;

    1 reference
    public void Disparar(
        Producto producto)
    {
        StockMinimo?.Invoke(
            $"ALERTA: stock mínimo de {producto.Nombre}");
    }

    2 references
    public void NotificarStockMinimo(
        Producto producto)
    {
        Disparar(producto);
    }
}

```

y las clases EventoX existentes ahora las implementan (agregaron un método `NotificarX(...)` que internamente llama al mismo `Disparar(...)` de siempre, por eso el mensaje en consola no cambia). Los servicios reciben la interfaz por constructor en vez de crear el evento con `new`.

Esto demuestra que el comportamiento observable del original al corregido (de buenos principios SOLID) no cambió, solo la forma de invocarlo entonces, el servicio dejó de crear sus propias dependencias con `new` y pasó a recibirlas inyectadas como interfaces segregadas. `EventoStockMinimo` implementa `INotificadorStockMinimo` delegando en su método `Disparar` original, por lo que el mensaje y el color en consola se preservan exactamente

Otra violación observada en el código original pero corregida en nuestro caso, fue SRP, en `ServicioUsuario` mezclaba carga, gestión y autenticación

```

public bool Login(
    string user,
    string password)
{
    return AspectoAutenticacion.Login(
        usuarios,
        user,
        password);
}

```

En el original, tenía el método Login(user, password) que llamaba a AspectoAutenticacion.Login(...), además de cargar usuarios desde archivo.

Violaba SRP porque la clase tenía dos razones de cambio distintas, si cambia la forma de guardar usuarios, o si cambia la forma de autenticar, ambas obligan a tocar la misma clase.

La solución aplicada fue crear ServicioAutenticacion, una clase nueva que recibe la lista de usuarios y expone Login(user, password) delegando en AspectoAutenticacion (que no se tocó). ServicioUsuario ahora solo expone ObtenerUsuarios().

```

public class ServicioAutenticacion
{
    private readonly List<Usuario> usuarios;

    public ServicioAutenticacion(
        List<Usuario> usuarios)
    {
        this.usuarios = usuarios;
    }

    public bool Login(
        string user,
        string password)
    {
        return AspectoAutenticacion.Login(
            usuarios,
            user,
            password);
    }
}

```

La responsabilidad de autenticar se extrajo a una clase independiente, ServicioAutenticacion, dejando a ServicioUsuario con una única razón de cambio: custodiar la colección de usuarios.

Otro comportamiento identificado, donde se incumplía con los principios SOLID, fue en IServicioNotificacion, porque contaba con un único método genérico EnviarNotificacion(string mensaje).

```

public interface IServicioNotificacion
{
    void EnviarNotificacion(string mensaje);
}

```

Interfaces

- IDescuento.cs
- IServicioNotificacion.cs

Para evitar el incumplimiento de ISP que es tener varios metodos en una sola interfaz, se implemento la solución creando 4 interfaces de un solo metodo cada una:

```

INotificadorMovimiento.cs
INotificadorPuntos.cs
INotificadorStockMinimo.cs
INotificadorVencimiento.cs

```

Se optó por cuatro interfaces segregadas de un solo método en vez de una única interfaz con los cuatro métodos, para que cada servicio dependa exclusivamente del contrato que usa.

Otro comportamiento identificado como violación a SOLID es OCP

```

MedicamentoCapsula medicamento =
    new MedicamentoCapsula(
        datos[0],
        decimal.Parse(datos[1]),
        int.Parse(datos[2]),
        int.Parse(datos[3]),
        DateTime.Parse(datos[4]),
        laboratorio,
        Enum.TipoRelleno.Gel);

productos.Add(medicamento);

return "Productos cargados";

```

Para soportar un cosmético, tocaba editar ese mismo método, mezclando el riesgo de romper la carga de medicamentos que ya funcionaba.

Se crearon las clases Cosmetico y Comestible, ambas heredando directamente de Producto (no de Medicamento, porque un cosmético no es un medicamento).

```

public class Cosmetico : Producto
{
    public string Marca { get; set; }

    public Cosmetico(
        string nombre,
        decimal precio,
        int stock,
        int stockMinimo,
        DateTime fechaVencimiento,
        string marca)
        : base(nombre, precio, stock,
            stockMinimo, fechaVencimiento)
    {
        Marca = marca;
    }
}

```

```

public class Comestible : Producto
{
    public int Calorias { get; set; }

    public Comestible(string nombre, decimal precio, int stock, int stockMinimo,
        DateTime fechaVencimiento, int calorias)
        : base(nombre, precio, stock,
            stockMinimo, fechaVencimiento)
    {
        Calorias = calorias;
    }
}

```

Se crearon RepositorioCosmeticoArchivo y RepositorioComestibleArchivo, cada uno implementando IRepositoryProducto y sabiendo construir únicamente su propio tipo.

```

public IEnumerable<Producto> LeerProductos(
    string ruta)
{
    string[] lineas = File.ReadAllLines(ruta);

    foreach (string linea in lineas)
    {
        string[] datos = linea.Split(';');

        yield return new Cosmetico(
            datos[0],
            decimal.Parse(datos[1]),
            int.Parse(datos[2]),
            int.Parse(datos[3]),
            DateTime.Parse(datos[4]),
            datos[5]);
    }
}

```

```

public interface IRepositoryProducto
{
    5 references
    bool Existe(string ruta);
    5 references
    IEnumerable<Producto> LeerProductos(string ruta);
}

```

ServicioProducto **no fue modificado en su lógica existente**. Se le agregó un método *nuevo* (sobrecarga): CargarDesdeArchivo(IRepositoryProducto repo, string ruta)

```

public string CargarDesdeArchivo(IRepositoryProducto repo, string ruta)
{
    try
    {
        if (!repo.Existe(ruta))
        {
            return "Archivo no encontrado";
        }

        foreach (var producto in
            repo.LeerProductos(ruta))
        {
            productos.Add(producto);
        }
    }
}

```

En Program.cs, se agregaron 2 líneas: instanciar los nuevos repos y llamar servicioProducto.CargarDesdeArchivo(repoCosmetico, "cosmeticos.txt") / lo mismo para comestibles.

```

RepositorioCosmeticoArchivo repoCosmetico =
    new RepositorioCosmeticoArchivo();

RepositorioComestibleArchivo repoComestible =
    new RepositorioComestibleArchivo();

Console.WriteLine(
    servicioProducto.CargarDesdeArchivo(
        repoCosmetico,
        "cosmeticos.txt"));

Console.WriteLine(
    servicioProducto.CargarDesdeArchivo(
        repoComestible,
        "comestibles.txt"));

```

Métrica de clases

Arquitectura	Clases creadas	Clases modificadas
AS-IS (si se hubiera implementado SC-1 sobre el diseño original)	2 (Cosmetico, Comestible)	1 (ServicioProducto)
TO-BE (como quedó implementado)	4 (Cosmetico, Comestible, RepositorioCosmeticoArchivo, RepositorioComestibleArchivo)	2 (ServicioProducto, Program.cs)

En el diseño original, agregar cosméticos y comestibles habría implicado editar directamente el método `CargarDesdeArchivo()` de `ServicioProducto`, el mismo que ya construye los medicamentos, con el riesgo de dañar algo que ya estaba funcionando. En el rediseño, `ServicioProducto` también aparece como clase modificada, pero el cambio fue distinto: se le agregó un método nuevo (`CargarDesdeArchivo(IRepositorioProducto repo, string ruta)`) sin tocar ni una línea del método que ya existía. Lo único que se modificó fuera de eso fue `Program.cs`, y solo para instanciar los nuevos repositorios y llamarlos, sin agregar lógica de negocio ahí.

Esa es la diferencia real: en ambos casos se cuenta una clase modificada, pero en el AS-IS esa modificación es sobre código que ya estaba en uso, mientras que en el TO-BE es una adición que no interfiere con lo que ya funcionaba. Eso es lo que demuestra que el principio abierto/cerrado se cumplió: el sistema quedó abierto a nuevos tipos de producto, pero cerrado a modificaciones sobre lo que ya estaba probado.

Evidencia de implementación y preservación del comportamiento

Para esta etapa se analiza la salida en consola de la versión final del sistema, correspondiente al código rediseñado bajo principios SOLID con la implementación de la solicitud de cambio SC-1. Esta ejecución permite observar que los comportamientos principales identificados en el sistema original se mantienen después del rediseño, al mismo tiempo que se evidencia la incorporación de las nuevas funcionalidades autorizadas por SC-1.

La principal diferencia observable respecto al sistema original corresponde a la incorporación de nuevos tipos de producto: cosméticos y comestibles. Esta diferencia no representa una alteración de los comportamientos existentes, sino la extensión funcional solicitada mediante SC-1.

Caso 1:

Carga de productos

```
ALERTA: stock mínimo de Dolex
ALERTA: stock mínimo de Amoxicilina
ALERTA: stock mínimo de Loratadina
ALERTA: stock mínimo de Diclofenaco
ALERTA: Dolex próximo a vencer
ALERTA: Ibuprofeno próximo a vencer
ALERTA: Amoxicilina próximo a vencer
ALERTA: Acetaminofen próximo a vencer
ALERTA: Omeprazol próximo a vencer
ALERTA: Loratadina próximo a vencer
ALERTA: VitaminaC próximo a vencer
ALERTA: Diclofenaco próximo a vencer
ALERTA: Aspirina próximo a vencer
ALERTA: Naproxeno próximo a vencer
ALERTA: ProtectorSolar próximo a vencer
ALERTA: CocaCola próximo a vencer
ALERTA: Snickers próximo a vencer
```

Correcto, trae la lista de productos de la fase 1, y los nuevos productos de cosmetica y comestibles.

Caso 2:

Listado

```
===== PRODUCTOS =====
Nombre      Stock  Precio
-----
Dolex       2      5000
Ibuprofeno  10     7000
Amoxicilina 1      12000
Acetaminofen 8      4500
Omeprazol   20     15000
Loratadina   4      9000
VitaminaC   15     6000
Diclofenaco  3      11000
Aspirina     9      7500
Naproxeno    7     13500
CremaNivea  20     15000
ShampooHead 15     12000
ProtectorSolar 8     25000
CocaCola    50     3500
Agua        100    1500
Snickers    30     2500
```

En la lista de productos, se evidencian los medicamentos pasados, con su stock y precio, y adicionalmente, salen los nuevos productos (CremaNivea, ShampooHead, ProtectorSolar, CocaCola, Agua, Snickers) con su stock y precio; en total hay 16 productos (10 medicamentos originales + 6 productos adquiridos de la nueva funcionalidad)

Caso 3: Búsqueda

En la regla de comportamiento (Fase 1 – AS-IS), se identifico que la búsqueda podía ser parcial, no distingue mayúsculas/minúscula, entonces como se buscará solo “crema” para verificar si encuentra CremaNivea

```
Ingrese nombre producto: crema
```

```
Producto: CremaNivea
```

```
Precio: 15000
```

```
Stock: 20
```

Coincide con la regla de minusculas y parcial.

```
Seleccione opción: 3
```

```
Ingrese nombre producto: Dolex
```

```
Producto: Dolex
```

```
Precio: 5000
```

```
Stock: 2
```

Asimismo, se busca el medicamento Dolex y evidenciamos el mismo comportamiento presentado en el AS-IS

Caso4: Búsqueda sin coincidencias

Buscar un nombre que no exista

```
Seleccione opción: 3
```

```
Ingrese nombre producto: creema
```

```
Producto no encontrado
```

Coincide con las pruebas realizadas en la fase 1, de que con alguna letra duplicada no hace una búsqueda a lo mas similar, sino que dice producto no encontrado.

Caso 5: Registro de venta

La entrada es seleccionar un producto existente y vender una cantidad válida

```
Seleccione opción: 4
```

```
Nombre producto: Agua
```

```
Cantidad: 10
```

```
Movimiento registrado: Venta
```

```
Venta registrada
```

Como resultado esperado se debe descontar la misma cantidad de stock y registrar el movimiento de forma equivalente:

Nombre	Stock	Precio

Dolex	2	5000
Ibuprofeno	10	7000
Amoxicilina	1	12000
Acetaminofen	8	4500
Omeprazol	20	15000
Loratadina	4	9000
VitaminaC	15	6000
Diclofenaco	3	11000
Aspirina	9	7500
Naproxeno	7	13500
CremaNivea	20	15000
ShampooHead	15	12000
ProtectorSolar	8	25000
CocaCola	50	3500
Agua	90	1500
Snickers	30	2500

Caso 6: Venta límite stock

Vender una cantidad igual al stock disponible

```
Nombre producto: Dolex
Cantidad: 6
Movimiento registrado: Venta
Venta registrada
```

Y en el stock se veía que tenía de límite 2

Como resultado esperado se debe conservar el comportamiento del sistema original respecto al stock y al movimiento generado.

Nombre	Stock	Precio

Dolex	-4	5000

Caso 7: Alerta de stock mínimo

Utilizar un producto cuyo stock sea igual o menor al stock mínimo

```
Seleccione opción: 6
Verificando alertas...
ALERTA: stock mínimo de Dolex
ALERTA: stock mínimo de Amoxicilina
ALERTA: stock mínimo de Loratadina
ALERTA: stock mínimo de Diclofenaco
```

Como resultado esperado se debe generar la misma alerta observable registrada en el AS-IS. Y efectivamente se cumple.

Caso 8: Alerta de vencimiento

Producto con fecha dentro del rango de alerta

```
ALERTA: Dolex próximo a vencer
ALERTA: Ibuprofeno próximo a vencer
ALERTA: Amoxicilina próximo a vencer
ALERTA: Acetaminofen próximo a vencer
ALERTA: Omeprazol próximo a vencer
ALERTA: Loratadina próximo a vencer
ALERTA: VitaminaC próximo a vencer
ALERTA: Diclofenaco próximo a vencer
ALERTA: Aspirina próximo a vencer
ALERTA: Naproxeno próximo a vencer
ALERTA: ProtectorSolar próximo a vencer
ALERTA: CocaCola próximo a vencer
ALERTA: Snickers próximo a vencer
```

Misma alerta observable

Regla confirmada: días ≤ 30 dispara alerta amarilla; el caso "exactamente 30" también dispara (Fase 1 lo verifiqué explícitamente)

Caso 9: Acumulación de puntos

Cliente existente, cantidad de puntos

```
Nombre cliente: Carlos
Puntos: 5
Cliente Carlos acumuló 5 puntos
```

Mismo total que en AS-IS puntos nuevos = puntos anteriores + puntos ingresados; búsqueda de cliente parcial y no distingue mayúsculas/minúsculas

(confirmado con Carlos/carlo en Fase 1)

```
===== CLIENTES =====
Carlos - Puntos: 5
```

Caso 10: Autenticación

La entrada o condición son credenciales válidas, luego inválidas.

```
Cargando información del sistema...
Productos cargados
Productos cargados
Productos cargados
Clientes cargados
Usuarios cargados

===== LOGIN =====
Usuario: admin
Contraseña: 1234
Login correcto
```

```
===== LOGIN =====
Usuario: admin
Contraseña: 345
Acceso denegado
```

Mismo resultado en ambos intentos Válidas → Login correcto, continúa al menú. Inválidas → Acceso denegado, el programa termina

A través de todas estas pruebas realizadas para conocer e identificar los comportamientos, se evidencia que en los 10 casos probados, el sistema rediseñado conservó el comportamiento observado en la Fase 1, incluso en los casos donde ese comportamiento no era del todo correcto (como permitir vender sin validar el stock disponible), lo cual confirma que el rediseño no alteró ninguna regla de negocio existente, solo la forma en que el código está organizado internamente.

Tabla comparativa AS-IS vs TO-BE

ID	Funcionalidad	Resultado AS-IS (Fase 1)	Resultado TO-BE V1 (Fase 4)	¿Coincide?
CP-01	Carga de productos	Carga los 10 medicamentos desde productos.txt	Carga los mismos 10 medicamentos desde productos.txt, sin ningún error de lectura	Sí
CP-02	Listado de productos	Muestra nombre, stock y precio de cada producto	Muestra el mismo formato (nombre, stock y precio) para los 10 productos originales	Sí
CP-03	Búsqueda existente	Búsqueda parcial, no distingue mayúsculas/minúsculas	Mantiene la búsqueda parcial y no distingue mayúsculas/minúsculas. Se obtiene el mismo resultado que en AS-IS para la búsqueda realizada	Sí
CP-04	Búsqueda sin coincidencias	Si no hay coincidencia, muestra "Producto no encontrado"	Con el mismo nombre inexistente utilizado en AS-IS, responde igual y no busca aproximaciones	Sí
CP-05	Registro de venta	Descuenta stock y registra el movimiento correspondiente	Se descuenta el stock y se registra el movimiento de la misma forma	Sí
CP-06	Venta en límite de stock	El sistema permite vender toda la cantidad disponible sin bloquear la operación	Se vendió un producto con stock de 2, utilizando una cantidad igual a la disponible, y el sistema lo permitió sin ninguna restricción, igual que en el original	Sí
CP-07	Alerta de stock mínimo	Se genera alerta cuando $\text{Stock} \leq \text{StockMínimo}$, incluyendo el caso en que son iguales	Se generó la misma alerta para los productos que cumplen la condición	Sí
CP-08	Alerta de vencimiento	Se genera alerta cuando faltan 30 días o menos, incluyendo exactamente 30 días	Se generó la misma alerta, con el mismo criterio de días	Sí

CP-09	Acumulación de puntos	Los puntos nuevos se suman a los existentes; la búsqueda del cliente es parcial y no distingue mayúsculas	Mismo resultado: los puntos se acumulan sobre el total anterior y la búsqueda funciona igual	Sí
CP-10	Autenticación	Con credenciales válidas entra al menú; con inválidas se niega el acceso y termina el programa	Se probaron ambos casos y el resultado fue el mismo en los dos	Sí

La tabla anterior presenta la comparación formal de los casos de caracterización entre el sistema original (AS-IS) y la versión V1 del rediseño SOLID, sin incluir todavía la solicitud de cambio SC-1. Esta comparación permite aislar el efecto del rediseño y verificar la preservación de los comportamientos existentes. De manera complementaria, las evidencias de ejecución en consola, presentada en los casos analizados, corresponden a la versión final del sistema, que incorpora SC-1, permitiendo comprobar que dichos comportamientos se mantienen y que, adicionalmente, se incorporan los nuevos productos y comportamientos asociados a la solicitud de cambio.